mammoviz-tda

Guía de uso del entorno de trabajo

Noviembre 2026

Resumen

Este documento es una guía práctica, independiente de la memoria del TFG, para instalar el entorno, ejecutar el pipeline con datos reales de CBIS-DDSM y trabajar de forma interactiva en un notebook. No reemplaza la memoria (`tfg/main.pdf`): aquí solo se documenta *cómo usar el código*.

Índice

1. Visión general **1**
1.1. Estructura del repositorio 2

2. Instalación **2**

3. Comandos, en el orden en que se usan **2**
3.1. 1. Descargar los metadatos reales (CSV) 2
3.2. 2. Descargar imágenes reales (piloto incremental) 3
3.3. 3. Pipeline TDA completo 3
3.4. 4. CNN de referencia (PyTorch) 3
3.5. 5. Comparar TDA vs. CNN vs. fusión 3
3.6. 6. Validación de la hipótesis longitudinal (datos sintéticos) 4

4. Entorno de trabajo interactivo: notebook **4**
4.1. Si quieres encadenar varios notebooks 5

5. Problemas conocidos y cómo evitarlos **5**

1. Visión general

El proyecto combina Análisis Topológico de Datos (TDA) y *deep learning* sobre mamografías reales de CBIS-DDSM, con el objetivo de estudiar la detección anticipada de cáncer de mama mediante comparación longitudinal de exploraciones sucesivas (ver los issues del milestone `F0'` en GitHub para el contexto completo de esta hipótesis).

1.1. Estructura del repositorio

Carpeta/fichero	Contenido
`src/`	Librería: carga de datos, preprocesado, TDA, modelos, evaluación
`scripts/`	Ejecutables de un solo comando (ver Sección 3)
`notebooks/`	Entorno de trabajo interactivo (ver Sección 4)
`tests/`	Tests automáticos (`pytest`)
`data/`	Datos descargados (NO versionado en git; hay que descargarlos)
`results/`	Salidas de los scripts (métricas, figuras)
`tfg/`	Memoria del TFG en LaTeX
`bib/`	PDFs de la bibliografía

Importante — qué NO persiste entre sesiones. La carpeta `data/` está en `.gitignore` a propósito (163 GB en el dataset completo, imposible de versionar). Cada vez que empiece de cero en un equipo nuevo, o si la carpeta desaparece por limpieza/sincronización, hay que volver a ejecutar los comandos de descarga de la Sección 3.

2. Instalación

```
# 1. Entorno virtual
python -m venv .venv
.venv\Scripts\activate       # Windows
source .venv/bin/activate    # Linux/Mac

# 2. Dependencias
pip install -r requirements.txt

# 3. Verificar que todo funciona
pytest
```

`requirements.txt` incluye `gudhi`/`giotto-tda` (TDA), `torch`/`torchvision` (CNN), `pydicom`/`requests` (descarga de datos reales) y `pytest`. En Python 3.13+ `giotto-tda` no publica ruedas: el código detecta automáticamente qué backend TDA hay instalado (`src.topology.backend()`) y usa `gudhi` de respaldo.

Si `pytest` da 45 tests en verde (1 *skip* esperado si no tienes `giotto-tda`), el entorno está listo.

3. Comandos, en el orden en que se usan

3.1. 1. Descargar los metadatos reales (CSV)

Los 4 CSV de descripción de casos de CBIS-DDSM (con la etiqueta real benigno/maligno) se descargan directamente, sin necesidad de registro:

```
mkdir -p data/csv
```

```
curl -O https://www.cancerimagingarchive.net/wp-content/uploads/
    mass_case_description_train_set.csv
curl -O https://www.cancerimagingarchive.net/wp-content/uploads/
    mass_case_description_test_set.csv
curl -O https://www.cancerimagingarchive.net/wp-content/uploads/
    calc_case_description_train_set.csv
curl -O https://www.cancerimagingarchive.net/wp-content/uploads/
    calc_case_description_test_set.csv
# mover los 4 ficheros .csv descargados a data/csv/
```

3.2. 2. Descargar imágenes reales (piloto incremental)

El dataset completo de imágenes son 163,5 GB — demasiado para descargar de golpe. Este comando descarga un piloto balanceado benigno/maligno directamente contra la API REST de TCIA (no hace falta el NBIA Data Retriever), y se puede volver a lanzar cuantas veces se quiera: nunca repite descargas ya hechas, solo añade pacientes nuevos.

```
python scripts/download_cbis_ddsm.py --add 15 15 --category mass
```

`-add 15 15` añade 15 pacientes benignos y 15 malignos *nuevos*. Tarda varios minutos (cada mamografía completa son decenas de MB). Se puede relanzar en segundo plano y comprobar el progreso en el fichero de log.

3.3. 3. Pipeline TDA completo

Un único comando: carga datos reales → partición por paciente (sin fuga de datos) → preprocesado → homología persistente → clasificador → métricas clínicas.

```
python scripts/run_pipeline.py
python scripts/run_pipeline.py --classifier rf --n-bins 4 # valores por defecto
python scripts/run_pipeline.py --tune                  # ajusta hiperparametros por
    GridSearchCV
```

Guarda en `results/`: `confusion_matrix.png`, `persistence_diagram_example.png` y `metrics.txt` (incluye el AUC por validación cruzada, más fiable que el de un test pequeño).

3.4. 4. CNN de referencia (PyTorch)

```
python scripts/train_cnn.py --epochs 10
```

Entrena una DenseNet121 (base congelada, cabeza nueva) sobre las mismas imágenes y partición que el pipeline TDA. Semilla fija: resultados reproducibles entre ejecuciones.

3.5. 5. Comparar TDA vs. CNN vs. fusión

```
python scripts/fuse_and_evaluate.py
```

Entrena ambos modelos sobre los mismos casos y reporta accuracy/AUC (test y validación cruzada) para las 3 vías: TDA sola, CNN sola, fusión de características (`src.models.fuse_features`).

3.6. 6. Validación de la hipótesis longitudinal (datos sintéticos)

Mientras se gestiona el acceso a un dataset longitudinal real (CSAW-CC o EMBED, ambos bajo solicitud formal — ver issue `#92` en GitHub):

```
python scripts/synthetic_longitudinal_validation.py
python scripts/synthetic_longitudinal_validation.py --sweep # analisis de
    sensibilidad
```

4. Entorno de trabajo interactivo: notebook

Para explorar, depurar o visualizar resultados celda a celda (en vez de ejecutar un script de un tirón), usa:

`notebooks/02_real_data_pipeline.ipynb`

Reutiliza exactamente las mismas funciones que los scripts de la Sección 3 (`run_pipeline.py`, `train_cnn.py`, `synthetic_longitudinal_validation.py`), así que nunca hay dos implementaciones del mismo cálculo que puedan divergir. Las secciones del notebook son:

1. Imports y configuración.
2. Carga de metadatos reales + partición por paciente.
3. Visualización de un caso real (mamografía + máscara ROI).
4. Preprocesado y diagrama de persistencia de ese mismo caso.
5. Construcción del dataset completo (todos los casos ya descargados).
6. Clasificador topológico + métricas clínicas.
7. Validación por validación cruzada (más fiable que un test pequeño).
8. CNN de referencia + fusión (opcional, más lento).
9. Validación sintética de la hipótesis longitudinal.

```
jupyter lab notebooks/02_real_data_pipeline.ipynb
```

El requirements.txt instala `jupyterlab`, no el paquete antiguo `notebook`: el comando correcto es `jupyter lab`, no `jupyter notebook` (este último da "command not found" si no se instala también el paquete `notebook`).

El notebook original con datos sintéticos (`notebooks/01_pipeline_demo.ipynb`) sigue disponible para quien quiera ejecutar sin tener CBIS-DDSM descargado.

4.1. Si quieres encadenar varios notebooks

Para un flujo de trabajo más largo (p. ej. separar exploración de datos, experimentos de TDA y experimentos de CNN en notebooks distintos), la forma recomendada de *no* duplicar código es que cada notebook nuevo importe las mismas funciones compartidas, igual que hace `02_real_data_pipeline.ipynb`:

```
import sys
from pathlib import Path
sys.path.insert(0, str(Path.cwd().parent))     # raiz del repo
sys.path.insert(0, str(Path.cwd().parent / "scripts"))

import run_pipeline as rp
import train_cnn as tc
import synthetic_longitudinal_validation as slv
```

Así cada notebook es una *vista* distinta sobre el mismo pipeline, no una copia del código.

5. Problemas conocidos y cómo evitarlos

`data/` ha desaparecido. Normal (está en `.gitignore`). Repite los pasos 1 y 2 de la Sección 3.

Un script falla con `SyntaxError` en un marcador de conflicto de Git. El fichero quedó con un conflicto de Git sin resolver (pasó una vez con `scripts/run_pipeline.py`, causado por una herramienta de backup automático que hace commits sin supervisión). Busca y resuelve el conflicto a mano, o pide ayuda para resolverlo. Verifica que no quede ninguno con:

```
grep -rln "^<<<<<<<" --include="*.py" .
```

Backups automáticos revierten cambios recientes. Se ha observado que algún commit automático (`"Backup automatico: ..."`) restaura versiones antiguas de ficheros concretos. Si algo que arreglaste "vuelve.ª su estado anterior, no es un error tuyo: compruébalo con `git log -oneline - <fichero>` y vuelve a aplicar el cambio.

AUC muy distinto entre ejecuciones. Con pilotos pequeños (pocas decenas de pacientes), el AUC sobre un test de pocos casos es ruidoso. Usa siempre el AUC por validación cruzada que reportan los scripts, no solo el del test.